Silesian University of Technology

Faculty of Automatic Control, Electronics and Computer Science

Department of Graphics, Computer Vision and Digital Systems

Microprocessor and Embedded Systems – Laboratory 23

AVR Microcontrollers – part III

Timer&Counters&Interrupts

M.Sc. Eng Jarosław Paduch

Exercise (carried out in the laboratory or remotely) description:

Requirements:

Hardware: One computer PC

Software: Microchip Studio - version 7.0 and above.

You can complete the exercises on your own computer, but you must bring the Microchip Studio software with you. We do not install the software during the lab! Download the latest (version 7.0.2542 or later) Microchip Studio from the Microchip website (web or offline installer):

https://www.microchip.com/en-us/development-tools-tools-and-software/microchip-studio-for-avr-and-sam-devices

During the exercise, students are to:

- Use final code/project from laboratory 22- in Microchip STUDIO version 7.0 or above,

- Make the necessary changes to the program in accordance with the assigned task,

- Write, compile, and link the program,

- Observe program execution in the simulator, step by step.

- Prepare and test program execution with different data,

- Check the correct delay times in the simulator before programming the device hardware,

- Show the program working as intended to the teacher conducting the exercise,

If the exercise takes place in the laboratory, program the hardware- Arduino ATMEG2560 board, and check program execution in real hardware (the programming instruction are in the Instruction of Laboratory 22 in appendix 2),

- Analyse and note the memory size of the program,

- Prepare and send the report to the ZMITAC database.

Report

The report should contain:

- Proper Title page,

- The subject of the program task,

- Source code of the program in ASM language with comments!!! / Please do not include the all project solution/

- Description and results of comprehensive testing of the ASM program.

- Screenshots confirming that the required delay times have been achieved,

- Summary of code size for selected test (using example template).

- Conclusions!!!

The report instructions and template are on the website:

Distance Education Platform for Faculty of Automatic Control, Electronics and Computer Science: https://platforma.polsl.pl/rau2/login/index.php

📘 Goal

The purpose of the exercise is to familiarize with the architecture of AVR microcontrollers on the example of ATMEGA 2560. Students during the laboratory should:

1. Learn about the AVR microcontrollers interrupts,
2. Learn about the AVR microcontrollers timers/counters,
3. Write a simple "ASM" program given by the teacher, using 16-bit Timer1, program the device and test program behaviour,
4. Send a report.

ALL Programs assumptions:

- Processor type: ATMEGA2560

- The frequency of the processor clock is equal to 16 [MHz]

- The "TAB_ROM" table is in the program memory (FLASH). The table has unspecified length (it can be empty) but has the word "0000" at the end. The starting address of the table can be anywhere in memory. A word "0000" does not belong to the table and should not be copied.

- The "TAB_RAM" table is in the data memory (RAM). The table is **3 to 300** bytes long. TAB_RAM can't be empty.

- There are buttons and LEDs connected to different ports of microcontroller. Please examine the schematics (Instruction lab 22) to see which port bits are used and how.

-When displaying the contents of the table, bit "1" means that the diode is on and bit "0" means that the diode is off.

- Please analyze the circuit diagram of add-on board.

- All ports used in the circuit should be programmed as they are used in the diagram, the remaining (unused or unconnected) ports should be programmed as inputs without pull-up resistors,

- All buttons connected to microcontroller are "real" buttons and have contact vibrations; the vibration time constant is maximum 20ms. (see->Instruction lab 22)

Tasks To DO:

1. Start by turning off all diodes at the beginning of the program. Wait for a button press. The table copying process initiates when any button connected to port A is pressed. Write a program that copies each byte from TAB_ROM to TAB_RAM in sequence. If TAB_RAM is larger than TAB_ROM, the copying wraps around, so bytes from the start of TAB_ROM are copied again after reaching its end, until all of TAB_RAM is filled. Each byte copied should be shown on the diodes connected to port C. Include a delay of about 500ms between displaying each byte. ~~After completing the process, turn on the LED connected to port B.7.~~ After that: Turn LED connected to Port B.7 exactly: for 1000ms - ON, next 500ms - OFF, next for 1000ms - ON and then switch to OFF.

2. Please turn off all diodes at the start of the program. Wait until the button is pressed. Copying tables begins after pressing any button connected to port A. Write a program that copies each byte with an odd address from TAB_ROM to TAB_RAM individually. If TAB_RAM is larger than TAB_ROM, the program will continue copying bytes to the end of TAB_RAM, looping back to the start of TAB_ROM as needed. Each copied byte should be displayed on the diodes connected to port C. The delay between displaying each byte should be approximately 500ms. ~~Turn on the LED connected to port B.6 after the program finishes.~~ After that: Turn LED connected to Port B.6 exactly: for 1000ms - ON, next 500ms - OFF, next for 500ms - ON and then switch to OFF.

3. At the start of the program, turn off all diodes. Wait for the button press. Initiate table copying after any button connected to port A is pressed. Write a program to transfer bytes from TAB_ROM to TAB_RAM, copying one byte at each even address. If TAB_RAM exceeds TAB_ROM, continue copying from the beginning of TAB_ROM once its end is reached. Display each copied byte on the diodes linked to port C, with about a 500ms pause between each. ~~After completion, turn on the LED connected to port B.5.~~ After that: Turn LED connected to Port B.5 exactly: for 500ms - ON, next 1000ms - OFF, next for 500ms - ON and then switch to OFF.

4. Please turn off all diodes at the start of the program. Wait for the button to be pressed. Copying tables begins when any button on port A is pressed. Write a program that copies all bytes with odd values from TAB_ROM to TAB_RAM, one by one. If TAB_RAM is larger than TAB_ROM, the

program will continue copying bytes to the end of TAB_RAM, starting again from the beginning of TAB_ROM. Each copied byte should be displayed on the diodes connected to port C. The delay between displaying each byte should be approximately ~~500 ms~~. 250ms. ~~Turn on the LED connected to port B.7 after the program completes.~~ After that: Turn LED connected to Port B.7 exactly: for 1000ms - ON, next 500ms - OFF, next for 1000ms - ON and then switch to OFF.

5. Please turn off all diodes at the start of the program. Wait for the button to be pressed. Copying tables begins when any button on port A is pressed. Write a program that copies all even-valued bytes from TAB_ROM to TAB_RAM sequentially. If TAB_RAM is larger than TAB_ROM, the copying continues from the beginning of TAB_ROM once the end is reached, wrapping around as necessary. Display each copied byte on the diodes connected to port C. The delay between displaying each byte should be approximately ~~500 ms~~. 250ms. ~~Turn on the LED connected to port B.6 after the program has completed.~~ After that: Turn LED connected to Port B.6 exactly: for 1000ms - ON, next 500ms - OFF, next for 500ms - ON and then switch to OFF.

6. Begin by switching off all diodes at the start of the program. Wait until a button is pressed. The table copying process commences after any button connected to port A is pressed. Write a program that copies each byte greater than 0x80 from TAB_ROM to TAB_RAM, one byte at a time. If TAB_RAM is larger than TAB_ROM, the copying should continue from the start of TAB_ROM until TAB_RAM is fully filled. Display each byte on the diodes connected to port C, with approximately a ~~500 ms~~. 250ms. ~~-~~delay between displays. ~~Afterwards, turn on the LED connected to port B.5 to indicate completion.~~ After that: Turn LED connected to Port B.5 exactly: for 500ms - ON, next 1000ms - OFF, next for 500ms - ON and then switch to OFF.

7. Please turn off all diodes at the start of the program. Wait for a button press. Copying tables begins when any button on port A is pressed. Write a program that copies each byte less than 0x80 from TAB_ROM to TAB_RAM, one byte at a time. If TAB_RAM is larger than TAB_ROM, the copying continues from the beginning of TAB_ROM once the end is reached, wrapping around to fill TAB_RAM entirely. Display each copied byte on the diodes connected to port C. The interval between displaying each byte should be approximately ~~500ms~~ 1000ms. ~~Turn on the LED~~

~~connected to port B.7 after the program has finished.~~ After that: Turn LED connected to Port B.7 exactly: for 1000ms - ON, next 250ms - OFF, next for 1000ms - ON and then switch to OFF.

8. Initially, turn off all diodes. Wait for a button press, then copy tables when any button on port A is pressed. Write a program that copies each byte from TAB_ROM to TAB_RAM in sequence. If TAB_ROM is larger than TAB_RAM, the copying wraps around, so bytes from the start of TAB_RAM are copied again after reaching its end, until all of TAB_ROM is read. Display each copied byte on port C diodes, pausing about~~500ms~~ 1000ms. between each. ~~Afterwards, turn on the LED connected to port B.6.~~ After that: Turn LED connected to Port B.6 exactly: for 1000ms - ON, next 250ms - OFF, next for 500ms - ON and then switch to OFF.

9. Initially, turn off all diodes and wait for a button press. Table copying starts as soon as any button connected to port A is pressed. Write a program that copies each byte with an odd address from TAB_ROM to TAB_RAM individually. If TAB_ROM is larger than TAB_RAM, the program will continue copying bytes to the end of TAB_ROM, looping back to the start of TAB_RAM as needed. Each transferred byte should be Each transferred byte should be displayed on the diodes connected to port C, with approximately a ~~500ms~~ 1000ms. pause between each. ~~Finally, turn on the LED connected to port B.5 once the task is complete.~~ After that: Turn LED connected to Port B.5 exactly: for 250ms - ON, next 1000ms - OFF, next for 1000ms - ON and then switch to OFF.

10. Please turn off all diodes at the start of the program. Wait for a button press. Table copying will begin once any button connected to port A is pressed. Write a program that copies bytes from TAB_ROM to TAB_RAM, transferring one byte at each even address. If TAB_ROM is larger than TAB_RAM, the program will keep copying bytes to the end of TAB_ROM and then loop back to the start of TAB_RAM as necessary. Display each copied byte on the diodes connected to port C. Pause for about ~~500m~~ 750ms between displaying each byte. ~~Turn on the LED connected to port B.7 after the program finishes.~~ After that: Turn LED connected to Port B.7 exactly: for 750ms - ON, next 500ms - OFF, next for 750ms - ON and then switch to OFF.

11. Begin by turning off all diodes at the start of the program. Wait until a button is pressed. Copying tables commences once any button connected to port A is pressed. Write a program that copies all bytes with odd values from TAB_ROM to TAB_RAM, one by one. If TAB_ROM is larger than TAB_RAM, the program will continue copying bytes to the end of TAB_ROM, looping back to the start of TAB_RAM as necessary. Display each copied byte on the diodes connected to port C. The delay between displaying each byte should be approximately ~~500ms~~. 750ms. ~~Turn on the LED connected to port B.6 once the program finishes.~~ After that: Turn LED connected to Port B.6 exactly: for 250ms - ON, next 750ms - OFF, next for 250ms - ON and then switch to OFF.

12. Start by turning off all diodes at the beginning of the program. Wait until a button is pressed. The process of copying the table begins as soon as any button connected to port A is pressed. Write a program that sequentially copies all even-valued bytes from TAB_ROM to TAB_RAM. If TAB_ROM is larger than TAB_RAM, the copying continues at the end of TAB_ROM, looping back to the beginning of TAB_RAM when needed. Each byte copied should be displayed on the diodes connected to port C. Include a delay of about ~~500 ms~~ 750ms between displaying each byte. ~~Finally, turn on the LED connected to port B.5 after the program completes.~~ After that: Turn LED connected to Port B.5 exactly: for 750ms - ON, next 250ms - OFF, next for 750ms - ON and then switch to OFF.

13. Begin by turning off all diodes initially. Wait for a button press. When any button connected to port A is pressed, copying tables will commence. Write a program to transfer each byte over 0x80 from TAB_ROM to TAB_RAM, one at a time. If TAB_ROM is larger than TAB_RAM, continue copying until reaching the end of TAB_ROM, then loop back to the start of TAB_RAM for further copying. Display each copied byte on the diodes connected to port C. Include a delay of about 500ms between displaying each byte. ~~After completion, turn on the LED connected to port B.7.~~ After that: Turn LED connected to Port B.7 exactly: for 1000ms - ON, next 250ms - OFF, next for 500ms - ON and then switch to OFF.

14. Please turn off all diodes at the start of the program. Wait for the button to be pressed. Copying tables begins after pressing any button connected to port A. Write a program that copies each byte less than 0x80 from TAB_ROM to TAB_RAM, one byte at a time. If TAB_ROM is larger than

TAB_RAM, the copying continues at the end of TAB_ROM, looping back to the beginning of TAB_RAM when necessary. Each copied byte should be displayed on the diodes connected to port C. The delay between displaying each byte should be approximately 500 ms. ~~Turn on the LED connected to port B.6 after the program completes.~~ After that: Turn LED connected to Port B.6 exactly: for 500ms - ON, next 1000ms - OFF, next for 250ms - ON and then switch to OFF.

15. Start by turning off all diodes at the beginning of the program. Wait until any button connected to port A is pressed to start copying tables. The program copies each byte from TAB_ROM to TAB_RAM in order. If TAB_RAM is larger, the copying wraps around, copying bytes from the beginning of TAB_ROM again after reaching its end, until TAB_RAM is full. If TAB_ROM is larger, copying continues from its end and loops back to the start of TAB_RAM as needed. Display each copied byte on the diodes connected to port C, pausing about 500ms between each display. ~~After completing the process, turn on the LED connected to port B.5.~~ After that: Turn LED connected to Port B.5 exactly: for 250ms - ON, next 1000ms - OFF, next for 250ms - ON and then switch to OFF.

BASICS

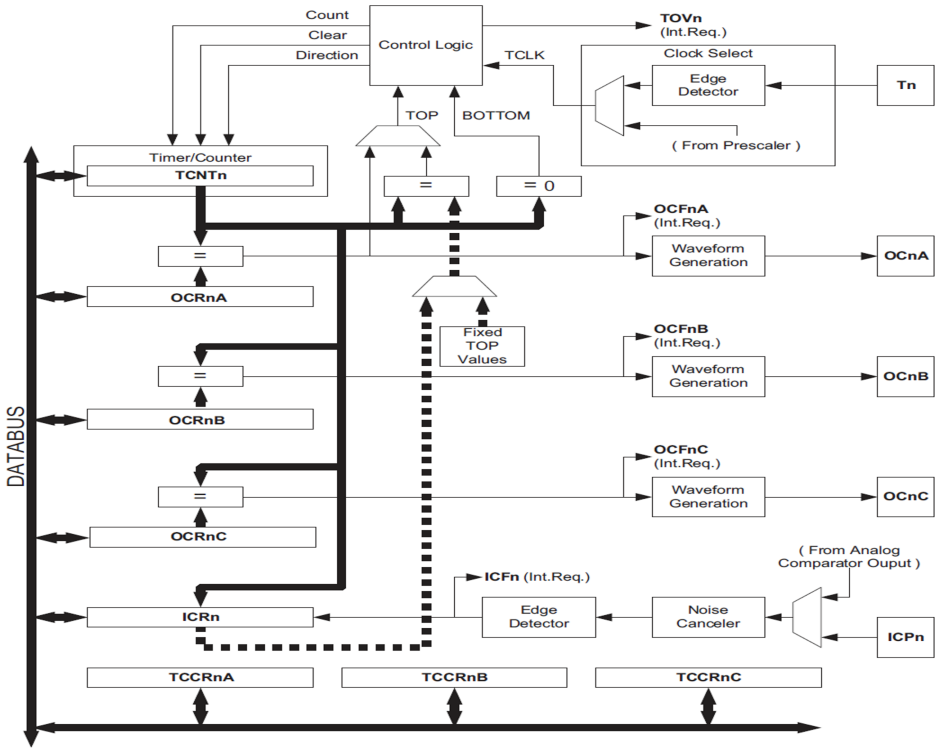
Timer/Counter 1

The 16-bit Timer/Counter 1 unit allows accurate program execution timing (event management), wave generation, and signal timing measurement. Timer 1 in ATmega2560 is a 16-bit timer/counter, meaning it can count from 0x0000 to 0xFFFF (0–65535). It can operate in multiple modes (Normal, CTC, PWM, etc.) and can use interrupts or be polled manually.

Features

- True 16-bit Design (that is, allows 16-bit PWM)
- Three independent Output Compare Units
- Double Buffered Output CompareRegisters
- One Input Capture Unit
- Input Capture Noise Canceler
- Clear Timer on Compare Match (Auto Reload)
- Glitch-free, Phase Correct Pulse Width Modulator (PWM)
- Variable PWM Period
- Frequency Generator
- External Event Counter
- Independent interrupt sources (TOV1, OCF1A, OCF1B, OCF1C, ICF1)

Timer/Counter Block diagram



Timer 1 Registers

Register	Description
TCCR1A	Timer/Counter1 Control Register A – selects mode bits (WGM10, WGM11) and output compare pins
TCCR1B	Timer/Counter1 Control Register B – contains prescaler bits (CS12..CS10) and mode bits (WGM12, WGM13)
TCNT1H / TCNT1L	Timer/Counter Register – current count value (16-bit, must be accessed as a pair)
OCR1A / OCR1B	Output Compare Registers – used for CTC (Compare Match) mode
TIMSK1	Timer Interrupt Mask Register – enables overflow or compare interrupts
TIFR1	Timer Interrupt Flag Register – holds flags for overflow or compare match
TCCR1C	Optional – controls forced compare (rarely used in simple timing)

TCNT1 Register

The **Timer/Counter1** – TCNT1 Register is as follows. It is 16 bits wide since the TIMER1 is a 16-bit register. **TCNT1H** represents the HIGH byte, whereas **TCNT1L** represents the LOW byte. The timer/counter value is stored in these bytes.

Bit	7	6	5	4	3	2	1	0	
	TCNT1[15:8]								TCNT1H
	TCNT1[7:0]								TCNT1L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR1A Register

Bit	7	6	5	4	3	2	1	0

(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	COM1C1	COM1C0	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:6 – COMnA1:0: Compare Output Mode for Channel A**
- **Bit 5:4 – COMnB1:0: Compare Output Mode for Channel B**
- **Bit 3:2 – COMnC1:0: Compare Output Mode for Channel C**

Compare Output Mode, non-PWM

COMnA1 COMnB1 COMnC1	COMnA0 COMnB0 COMnC0	Description
0	0	Normal port operation, OCnA/OCnB/OCnC disconnected
0	1	Toggle OCnA/OCnB/OCnC on compare match
1	0	Clear OCnA/OCnB/OCnC on compare match (set output to low level)
1	1	Set OCnA/OCnB/OCnC on compare match (set output to high level)

Compare Output Mode, Fast PWM

COMnA1 COMnB1 COMnC1	COMnA0 COMnB0 COMnC0	Description
0	0	Normal port operation, OCnA/OCnB/OCnC disconnected
0	1	WGM13:0 = 14 or 15: Toggle OC1A on Compare Match, OC1B and OC1C disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B/OC1C disconnected
1	0	Clear OCnA/OCnB/OCnC on compare match, set OCnA/OCnB/OCnC at BOTTOM (non-inverting mode)
1	1	Set OCnA/OCnB/OCnC on compare match, clear OCnA/OCnB/OCnC at BOTTOM (inverting mode)

Compare Output Mode, Phase Correct and Phase and Frequency Correct PWM

COMnA1 COMnB1 COMnC1	COMnA0 COMnB0 COMnC0	Description
0	0	Normal port operation, OCnA/OCnB/OCnC disconnected
0	1	WGM13:0 =9 or 11: Toggle OC1A on Compare Match, OC1B and OC1C disconnected (regular port operation). For all other WGM1 settings, regular port operation, OC1A/OC1B/OC1C disconnected
1	0	Clear OCnA/OCnB/OCnC on compare match when up-counting Set OCnA/OCnB/OCnC on compare match when down counting
1	1	Set OCnA/OCnB/OCnC on compare match when up-counting Clear OCnA/OCnB/OCnC on compare match when down counting

TCCR1B Register

The **Timer/Counter1 Control Register B**– TCCR1B Register is as follows.

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Right now, only the highlighted bits concern us. The **bit 2:0 – CS12:10** are the **Clock Select Bits** of TIMER1.

Their selection is as follows.

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}/1$ (No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

TCCR1C Register

Bit	7	6	5	4	3	2	1	0	
(0x82)	FOC1A	FOC1B	FOC1C	-	-	-	-	-	TCCR1C
Read/Write	W	W	W	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – FOCnA: Force Output Compare for Channel A**
- **Bit 6 – FOCnB: Force Output Compare for Channel B**
- **Bit 5 – FOCnC: Force Output Compare for Channel C**

TIMSK1 Register

The **Timer/Counter Interrupt Mask Register** – TIMSK Register is as follows.

TIMSK1 – Timer/Counter 1 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6F)	-	-	ICIE1	-	OCIE1C	OCIE1B	OCIE1A	TOIE1	TIMSK1
Read/Write	R	R	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The bits associated with other timers are greyed out. **Bits not greyed out** correspond to TIMER1. **Bit 0 – TOIE1 – Timer/Counter1 Overflow Interrupt Enable** bit enables the overflow interrupt of TIMER1.

TIFR Register

The **Timer/Counter Interrupt Flag Register** – TIFR is as follows.

TIFR1 – Timer/Counter1 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x16 (0x36)	–	–	ICF1	–	OCF1C	OCF1B	OCF1A	TOV1	TIFR1
Read/Write	R	R	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The greyed out bits correspond to different timers. Bits not greyed out are related to TIMER1. Of these, we are interested in **Bit 0 – TOV1 – Timer/Counter1 Overflow Flag**. This bit is set to '1' whenever the timer overflows. It is cleared (to zero) automatically as soon as the corresponding Interrupt Service Routine (ISR) is executed. Alternatively, if there is no ISR to run, we can clear it by writing '1' to it.

OCR1AH and OCR1AL Register

Bit	7	6	5	4	3	2	1	0	
(0x89)	OCR1A[15:8]								OCR1AH
(0x88)	OCR1A[7:0]								OCR1AL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ICR3H and ICR3L – Input Capture Register 3

Bit	7	6	5	4	3	2	1	0	
(0x97)	ICR3[15:8]								ICR3H
(0x96)	ICR3[7:0]								ICR3L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

The Timer/Counter (TCNTn), Output Compare Registers (OCRnA/B/C), and Input Capture Register (ICRn) are all 16-bit registers. Special procedures must be followed when accessing the 16-bit registers. The Timer/Counter Control Registers (TCCRnA/B/C) are 8-bit registers and have no CPU access restrictions. Interrupt requests (shortened as Int.Req.) signals are all visible in the Timer Interrupt Flag Register (TIFRn). All interrupts are individually masked with the Timer Interrupt Mask Register (TIMSKn). TIFRn and TIMSKn are not shown in the figure since other timer units share these registers.

The Timer/Counter can be clocked internally, via the prescaler, or by an external clock source on the Tn pin. The Clock Select logic block controls which clock source and edge the Timer/Counter uses to increment (or decrement) its value. The Timer/Counter is inactive when no clock source is selected. The output from the clock select logic is referred to as the timer clock (clk_{Tn}).

The double buffered Output Compare Registers (OCRnA/B/C) are compared with the Timer/Counter

value at all times. The Waveform Generator can use the compare result to generate a PWM or variable frequency output on the Output Compare pin (OCnA/B/C). The compare match event will also set the Compare Match Flag (OCFnA/B/C), which can generate an Output Compare interrupt request.

The Input Capture Register can capture the Timer/Counter value at a given external (edge triggered) event on the Input Capture pin (ICPn) or the Analog Comparator pins. The Input Capture unit includes a digital filtering unit (Noise Canceler) for reducing the chance of capturing noise spikes.

The TOP value, or maximum Timer/Counter value, can in some modes of operation be defined by either the OCRnA Register, the ICRn Register, or by a set of fixed values. When using OCRnA as a TOP value in a PWM mode, the OCRnA Register can not generate a PWM output. However, the TOP value will, in this case, be double buffered, allowing the TOP value to be changed in run time. If a fixed TOP value is required, the ICRn Register can be used as an alternative, freeing the OCRnA to be used as PWM output.

Accessing 16-bit Registers

The TCNTn, OCRnA/B/C, and ICRn are 16-bit registers that the AVR CPU can access via the 8-bit data bus. The 16-bit register must be byte accessed using two reads or write operations. Each 16-bit timer has a single 8- bit register to temporarily store the high byte of the 16-bit access. The same Temporary Register is shared between all 16-bit registers within each 16-bit timer. Accessing the low byte triggers the 16-bit read or write operation. When the CPU writes the low byte of a 16-bit register, the high byte stored in the Temporary Register, and the written low byte are both copied into the 16-bit register in the same clock cycle. When the CPU reads the low byte of a 16- bit register, the high byte of the 16-bit register is copied into the Temporary Register in the same clock cycle as the low byte is read.

Not all 16-bit accesses uses the Temporary Register for the high byte. Reading the OCRnA/B/C 16-bit registers does not involve using the Temporary Register.

To do a 16-bit write, the high byte must be written before the low byte. For a 16-bit read, the low byte must be read before the high byte.

The following code examples show how to access the 16-bit timer registers assuming that no interrupts update the temporary register. The same principle can be used directly for accessing the OCRnA/B/C and ICRn Registers. Note that when using "C", the compiler handles the 16-bit access. It is crucial to notice that accessing 16-bit registers are an atomic operation. Suppose an interrupt occurs between the two instructions accessing the 16-bit register, and the interrupt code updates the temporary register by accessing the same or any other of the 16-bit Timer Registers. In that case, the result of the access outside the interrupt will be corrupted. Therefore, when both the main code and the interrupt code update the temporary register, the main code must disable the interrupts during the 16-bit access.

1️⃣ Classic method (OUT – LOW the (most common, fastest for I/O)
For I/O space registers (TCNT1L/H, OCR1AL/H, ICR1L/H):

```
ldi r16, HIGH(0xE7A6)
out TCNT1H, r16
ldi r16, LOW(0xE7A6)
out TCNT1L, r16

or when timer is working

cli
ldi r16, HIGH(0xE7A6)
out TCNT1H, r16
ldi r16, LOW(0xE7A6)
out TCNT1L, r16
sei
```

🔑 Key Notes for Students

- Cycles:
  - LDI ×2 = 2
  - OUT ×2 = 2
- ➡️ Total: 4 cycles/6cycles (with cli/sei)
- ✔️ The fastest method
- ❌ only for I/O

2️⃣ Writing via STS (no address restrictions) It is possible because STS also works for peripheral registers, as they are also visible in the data space. Example:

```
ldi r16, HIGH(0xE7A6)
sts TCNT1H, r16
ldi r16, LOW(0xE7A6)
sts TCNT1L, r16
```

🔑 Key Notes for Students

- Cycles:
  - LDI ×2 = 2
  - STS ×2 = 4
- ➡️ Total: 6 cycles
- ✔️ universal
- ✔️ readable
- ❌ slower

3️⃣ Writing through index registers (X / Y / Z) This is really old school, but it can be very elegant, especially with register arrays.

Example: Writing TCNT1 through Z

```
ldi ZH, HIGH(TCNT1H)
ldi ZL, LOW(TCNT1H)

ldi r16, HIGH(0xE7A6)
st Z+, r16        ; write TCNT1H
ldi r16, LOW(0xE7A6)
st Z, r16         ; write TCNT1L
```

🔑 Key Notes for Students

- Works with both SRAM and I/O mapped registers
- Great for loops and macros
- Cycles:
  - LDI ×4 = 4
  - ST ×2 = 4
- ➡️ Total: 8 cycles
- 📌 if Z is already set:
- ➡️ only 4 cycles (LDI ×2 + ST ×2)

4️⃣ Writing 16 bits from a register pair (MOVW + STS)
The AVR doesn't have a "write word to memory" instruction,
but you can store a value in a register pair.

```
; r25:r24 = 0xE7A6
ldi r25, HIGH(0xE7A6)
ldi r24, LOW(0xE7A6)

sts TCNT1H, r25
sts TCNT1L, r24

albo

ldi ZH, HIGH(TCNT1H)
ldi ZL, LOW(TCNT1H)

st Z+, r25
st Z,  r24
```

🔑 Key Notes for Students

- Cycles:
  - LDI ×2 = 2
  - STS ×2 = 4
- ➡️ Total: 6 cycles
- ✔️ Ideal for 16-bit calculations
- ✔️ Very readable especially when counting 16-bit values passing them between functions

5️⃣ Macros for advanced programmers

```
Define:
.macro WRITE16 regH, regL, value
   ldi \regH, HIGH(\value)
   ldi \regL, LOW(\value)
.endm

Use:
WRITE16 r25, r24, 0xE7A6
sts TCNT1H, r25
sts TCNT1L, r24
```

or I/O write:

```
Define:
.macro IO_WRITE16 ioH, ioL, value
   ldi r16, HIGH(\value)
   out \ioH, r16
   ldi r16, LOW(\value)
   out \ioL, r16
.endm

Use:
IO_WRITE16 TCNT1H, TCNT1L, 0xE7A6
```

🔑 Key Notes for Students

- Cycles:
  - LDI ×2 = 2
  - STS ×2 = 4 or OUTx2=4
- ➡️ Total: 6 cycles

📘 Using Timer1 without Interrupts (Polling)

We'll toggle an LED on PORTC.0 every 100 ms using two modes:

1. Overflow mode
2. CTC (OCR1A compare) mode

Let's assume:

- CPU clock = 16 MHz
- Desired period = 100 ms
- LED toggles every 100 ms → full cycle 200 ms.

🧮 Calculations

Timer frequency depends on the prescaler:

$$f_{timer} = \frac{f_{CPU}}{prescaler}$$

The number of ticks per 100 ms:

$$N = f_{timer} \times 0.1$$

We must keep $N \leq 65536$.

Example with prescaler = 256:

$$f_{timer} = 16000000 / 256 = 62500 \text{ Hz}$$

$$N = 62500 * 0.1 = 6250 \text{ counts per 100 ms}$$

Setup Your Timer1

How to Calculate Timer Values (General Method)

For a desired time **T** (in seconds):

$$N = \frac{T \times f_{CPU}}{Prescaler}$$

Then depending on mode:

- **Normal mode:**

TCNT_start = 65536 - N

- **CTC mode:**

OCR1A = N - 1

To select a prescaler, find one that keeps $N \leq 65536$.

Example for 100 ms (T = 0.1 s, f_CPU = 16 MHz)

Prescaler	N	Fits 16-bit?
1	1 600 000	❌ Too big
8	200 000	❌ Too big
64	25 000	✅ OK
256	6250	✅ OK
1024	1562.5	✅ OK

Example Errors

For integer N only:

T_{real} = \frac{Prescaler \times (OCR1A + 1)}{f_{CPU}}

Error = \frac{T_{real} - T_{desired}}{T_{desired}} \times 100\%

For 256 prescaler, OCR1A = 6249:

T_{real} = 256 × (6249+1) / 16 000 000 = 0.10000 s

✅ practically zero error.

```
// lets turn on 16 bit timer1 with prescaler  /1024
TCCR1B |= (1 << CS10) | (1 << CS12);
```

$$Timer\ Count = \frac{Required\ Delay}{Clock\ Time\ Period} - 1$$

(A) Overflow Mode — No Interrupts

In Normal mode, the counter counts up to 0xFFFF, then sets the overflow flag TOV1.

We preload TCNT1 with:

TCNT1_{start} = 65536 - 6250 = 59286

Example Code in C (Polling Overflow)

```
#include <avr/io.h>

int main(void)
{
   DDRC |= (1 << PC0);  // LED as output
   TCCR1A = 0;          // Normal mode
   TCCR1B = (1 << CS12); // Prescaler 256
   TCNT1 = 65536 - 6250; // preload for 100ms

   while (1)
   {
      if (TIFR1 & (1 << TOV1))   // check overflow flag
      {
         TIFR1 |= (1 << TOV1);  // clear flag by writing 1
         PORTC ^= (1 << PC0);   // toggle LED
         TCNT1 = 65536 - 6250;  // reload
      }
   }
}
```

Example Code in ASM (Polling Overflow)

```
.include "m2560def.inc"

.org 0x0000
   rjmp RESET

RESET:
   ; Stack
   ldi r16, HIGH(RAMEND)
   out SPH, r16
   ldi r16, LOW(RAMEND)
   out SPL, r16

   ; PORTC.0 as output
   sbi DDRC, 0

   ; Timer1 normal mode
   clr r16
   out TCCR1A, r16
   ldi r16, (1<<CS12)       ; prescaler 256
   out TCCR1B, r16

   ; preload TCNT1 = 0xE7A6
   ldi r16, HIGH(0xE7A6)
   out TCNT1H, r16
   ldi r16, LOW(0xE7A6)
   out TCNT1L, r16

MAIN:
   sbis TIFR1, TOV1         ; overflow occurred?
   rjmp MAIN

   ; clear overflow flag
   sbi TIFR1, TOV1

   ; toggle LED
   sbi PINC, 0

   ; reload counter
   ldi r16, HIGH(0xE7A6)
   out TCNT1H, r16
   ldi r16, LOW(0xE7A6)
   out TCNT1L, r16

   rjmp MAIN
```

(B) CTC (OCR1A) Mode — No Interrupts

Set CTC mode (WGM12=1), and choose a suitable OCR1A value.

For prescaler = 256:
OCR1A = N - 1 = 6250 - 1 = 6249

Example Code in C (Polling Overflow)

```
#include <avr/io.h>
int main(void)
{
   DDRC |= (1 << PC0);
   TCCR1A = 0;
   TCCR1B = (1 << WGM12) | (1 << CS12); // CTC, prescaler 256
   OCR1A = 6249;

   while (1)
   {
      if (TIFR1 & (1 << OCF1A))
      {
         TIFR1 |= (1 << OCF1A); // clear flag
         PORTC ^= (1 << PC0);   // toggle LED
      }
   }
}
```

Example Code in ASM (Polling Overflow)

```
.include "m2560def.inc"

.org 0x0000
   rjmp RESET

RESET:
   ; Stack
   ldi r16, HIGH(RAMEND)
   out SPH, r16
   ldi r16, LOW(RAMEND)
   out SPL, r16

   ; LED output
   sbi DDRC, 0

   ; CTC mode, prescaler 256
   clr r16
   out TCCR1A, r16
   ldi r16, (1<<WGM12)|(1<<CS12)
   out TCCR1B, r16

   ; OCR1A = 6249
   ldi r16, HIGH(0x1869)
   out OCR1AH, r16
   ldi r16, LOW(0x1869)
   out OCR1AL, r16

MAIN:
   sbis TIFR1, OCF1A
   rjmp MAIN

   ; clear compare flag
   sbi TIFR1, OCF1A

   ; toggle LED
   sbi PINC, 0

   rjmp MAIN
```

⚡ 3. Using Timer 1 With Interrupts

Timer 1 Interrupts

Timer interrupts are an excellent way of having your AVR do something at a given interval. They can fire off and interrupt whatever the AVR is doing, making for exact timing. They are one of the best ways to implement custom waveforms for things such as positioning robot servos, dimming LED's, and driving speakers at different frequencies. In our laboratory, we concentrate on using Timer 1 only.

There are also other interrupt vectors available for timer 1:

TIMER1 CAPT	Timer/Counter1 Capture Event
TIMER1 COMPA	Timer/Counter1 Compare Match A
TIMER1 COMPB	Timer/Counter1 Compare Match B
TIMER1 COMPC	Timer/Counter1 Compare Match C
TIMER1 OVF	Timer/Counter1 Overflow

Turning on the Timer Interrupt

For the interrupt to fire, you must enable it in the TIMSK register. For timer1 interrupts, use the following code in main:

```
// enable timer overflow interrupt for both Timer1

TIMSK= (1<<TOIE1);
```

Now, we'll let the timer interrupt set a **flag**, and the **main loop** will poll that flag to toggle the LED.

C Example - Overflow Mode with Interrupt Flag

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t timer_flag = 0;

ISR(TIMER1_OVF_vect)
{
   timer_flag = 1;
   TCNT1 = 65536 - 6250; // reload
}

int main(void)
{
   DDRC |= (1 << PC0);

   TCCR1A = 0;
   TCCR1B = (1 << CS12); // Normal mode, prescaler 256
   TCNT1 = 65536 - 6250;
   TIMSK1 |= (1 << TOIE1); // enable overflow interrupt

   sei(); // enable global interrupts

   while (1)
   {
      if (timer_flag)
      {
         timer_flag = 0;
         PORTC ^= (1 << PC0);
      }
   }
}
```

C Example - CTC (OCR1A) Mode with Interrupt Flag

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t timer_flag = 0;

ISR(TIMER1_COMPA_vect)
{
   timer_flag = 1;
}

int main(void)
{
   DDRC |= (1 << PC0);

   TCCR1A = 0;
   TCCR1B = (1 << WGM12) | (1 << CS12); // CTC mode, prescaler 256
   OCR1A = 6249;
   TIMSK1 |= (1 << OCIE1A); // enable compare interrupt

   sei();

   while (1)
   {
      if (timer_flag)
      {
         timer_flag = 0;
         PORTC ^= (1 << PC0);
      }
   }
}
```

C Example (Overflow Mode)

Here is a simple example that turns on timer1. It accomplishes the following:

Sets up timer1 in a divide by 1024 mode, counting from 0 to 65,535

On timer1 interrupt, toggles PORTA bit 1.

You will see PORTA bit 1 blinking on and off every 8.3 seconds.

```
/*****************************************************************************
Includes
*****************************************************************************/
#include <avr/io.h>
#include <stdbool.h>
#include <avr/interrupt.h>
/*****************************************************************************
Interrupt Routines
*****************************************************************************/
// timer1 overflow
```

```
ISR(TIMER1_OVF_vect) {
   // XOR PORTA with 0x02 to toggle the LSB
   PORTA=PORTA ^ 0x02;
}
/*****************************************************************************
Main
*****************************************************************************/
int main( void ) {
   // Configure PORTA as output
   DDRA = 0xFF;
   PORTA = 0xFF;

   // enable timer overflow interrupt for Timer1
   TIMSK= (1<<TOIE1);
   // lets turn on 16 bit timer1 also with /1024
   TCCR1B |= (1 << CS10) | (1 << CS12);
   // enable interrupts
   sei();
   while(true) {
   }
}
```

C example (Compare Mode)

Here is a simple example that turns on timer1. It accomplishes the following:

Sets up timer1 to generate 4Hz blink frequency

On timer1 interrupt, toggles PORTA bit 1.

You will see PORTA bit 1 blinking on and off with 4HZ frequency

```
/*****************************************************************************
Includes
*****************************************************************************/
#include <avr/io.h>
#include <stdbool.h>
#include <avr/interrupt.h>

void setupTimer1() {
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;
```

```
  // 4 Hz (16000000/((15624+1)*256))
  OCR1A = 15624;
  // CTC
  TCCR1B |= (1 << WGM12);
  // Prescaler 256
  TCCR1B |= (1 << CS12);
  // Output Compare Match A Interrupt Enable
  TIMSK1 |= (1 << OCIE1A);
}

ISR(TIMER1_COMPA_vect) {
   // XOR PORTA with 0x02 to toggle the LSB
   PORTA=PORTA ^ 0x02;
}
int main( void ) {
   // Configure PORTA as output
   DDRA = 0xFF;
   PORTA = 0xFF;
   setupTimer1();
   sei();
   while(true) {
   }
```

ASM Example Timer1 Overflow Interrupt + Flag + LED Toggle (100 ms)

```
.include "m2560def.inc"

; -------------------------
; Constants
; -------------------------
.equ TCNT1_PRELOAD = 0xE7A6   ; 65536 - 6250 (100 ms @ 16 MHz, /256)

; -------------------------
; Register aliases
; -------------------------
.def temp = r16
.def flag = r17

; -------------------------
; Reset vector
; -------------------------
.org 0x0000
   rjmp RESET

; -------------------------
; Timer1 Overflow interrupt vector
; -------------------------
.org TIMER1_OVF_vect
TIMER1_OVF_ISR:
   ; Set software flag (main loop will act on it)
   ldi flag, 1
```

```
    ; Reload TCNT1 for the next 100 ms interval
    ; IMPORTANT: write HIGH first, then LOW (latches 16-bit value)
    ldi temp, HIGH(TCNT1_PRELOAD)
    out TCNT1H, temp
    ldi temp, LOW(TCNT1_PRELOAD)
    out TCNT1L, temp

    reti

; -------------------------
; Main program
; -------------------------
RESET:
    ; Initialize stack pointer
    ldi temp, HIGH(RAMEND)
    out SPH, temp
    ldi temp, LOW(RAMEND)
    out SPL, temp

    ; LED on PORTC.0 as output
    sbi DDRC, 0

    ; Clear software flag
    clr flag

    ; ---- Timer1 setup: Normal mode, prescaler = 256 ----
    ; Normal mode: WGM13..0 = 0 => TCCR1A=0 and TCCR1B WGM12/13=0
    clr temp
    out TCCR1A, temp

    ; Prescaler /256 => CS12=1, CS11=0, CS10=0
    ldi temp, (1<<CS12)
    out TCCR1B, temp

    ; Preload TCNT1 for 100 ms
    ldi temp, HIGH(TCNT1_PRELOAD)
    out TCNT1H, temp
    ldi temp, LOW(TCNT1_PRELOAD)
    out TCNT1L, temp

    ; (Good practice) clear pending overflow flag
    sbi TIFR1, TOV1

    ; Enable Timer1 overflow interrupt
    ldi temp, (1<<TOIE1)
    out TIMSK1, temp

    ; Enable global interrupts
    sei

; -------------------------
; Main loop: poll flag
; -------------------------
MAIN:
    tst flag
    breq MAIN           ; wait for ISR event

    clr flag            ; consume event
    sbi PINC, 0         ; toggle LED on PC0 (write 1 to PINC bit)

    rjmp MAIN
```

ASM Example - Timer1 CTC + Compare A with Interrupt

```
.include "m2560def.inc"

; -------------------------
; Configuration constants
; -------------------------
.equ OCR1A_100MS = 0x1869    ; 6249 decimal (100 ms at 16MHz, prescaler 256)

; -------------------------
; Register aliases
; -------------------------
.def temp = r16
.def flag = r17

; -------------------------
; Reset vector
; -------------------------
.org 0x0000
   rjmp RESET

; -------------------------
; Timer1 Compare A interrupt vector
; -------------------------
.org TIMER1_COMPA_vect
TIMER1_COMPA_ISR:
   ldi flag, 1          ; set software flag
   reti

; -------------------------
; Main program
; -------------------------
RESET:
   ; Initialize stack pointer
   ldi temp, HIGH(RAMEND)
   out SPH, temp
   ldi temp, LOW(RAMEND)
   out SPL, temp

   ; LED on PORTC.0 as output
   sbi DDRC, 0

   ; Clear flag variable
   clr flag

   ; ---- Timer1 setup: CTC mode, prescaler = 256 ----
   ; TCCR1A = 0
   clr temp
   out TCCR1A, temp

   ; TCCR1B: WGM12=1 (CTC), CS12=1 (prescaler 256)
   ldi temp, (1<<WGM12)|(1<<CS12)
   out TCCR1B, temp

   ; OCR1A = 6249 (0x1869)  -> write HIGH then LOW
   ldi temp, HIGH(OCR1A_100MS)
   out OCR1AH, temp
   ldi temp, LOW(OCR1A_100MS)
   out OCR1AL, temp

   ; (Good practice) clear pending compare flag
   sbi TIFR1, OCF1A

   ; Enable Timer1 Compare A interrupt
```

```
  ldi temp, (1<<OCIE1A)
  out TIMSK1, temp

  ; Enable global interrupts
  sei

; -------------------------
; Main loop: poll flag
; -------------------------
MAIN:
  tst flag
  breq MAIN          ; wait until ISR sets flag

  clr flag           ; consume event
  sbi PINC, 0        ; toggle LED on PC0 (write 1 to PINC bit)

  rjmp MAIN
```

ASM example CTC Mode with Polling

```
.include "m2560def.inc"

; --- rejestry robocze ---
.def temp     = r16
.def tick_cnt = r18     ; licznik ticków w bieżącej fazie
.def phase    = r19     ; 0..3

; OCR1A dla 250ms przy /256: 15624 = 0x3D08
.equ OCR_250MS = 0x3D08

.org 0x0000
   rjmp RESET

RESET:
   ; --- init stack ---
   ldi temp, HIGH(RAMEND)
   out SPH, temp
   ldi temp, LOW(RAMEND)
   out SPL, temp

   ; --- LED: PORTC.0 output ---
   sbi DDRC, 0

   ; --- Timer1: CTC, prescaler 256 ---
   clr temp
   out TCCR1A, temp
   ldi temp, (1<<WGM12) | (1<<CS12)     ; CTC + /256
   out TCCR1B, temp
```

```
.org TIMER1_OVF_vect
   rjmp TIMER1_OVF_ISR

.def temp = r16
.def flag = r17

.org 0x0000
   rjmp RESET

.org TIMER1_OVF_vect
TIMER1_OVF_ISR:
   ldi flag, 1

   ; reload timer
   ldi temp, HIGH(0xE7A6)
   out TCNT1H, temp
   ldi temp, LOW(0xE7A6)
   out TCNT1L, temp
   reti
```

```
MAIN:
   ; --- polling: czekaj na compare match ---
WAIT_TICK:
   sbis TIFR1, OCF1A
   rjmp WAIT_TICK

   ; skasuj flagę przez zapis 1
   sbi TIFR1, OCF1A

   ; mamy tick 250ms
   inc tick_cnt

   ; wybierz próg zależnie od fazy: 1,2,1,4
   cpi phase, 0
   breq PH0
   cpi phase, 1
   breq PH1
   cpi phase, 2
   breq PH2
   rjmp PH3

PH0:
   cpi tick_cnt, 1
   rjmp CHECK
PH1:
   cpi tick_cnt, 2
   rjmp CHECK
PH2:
   cpi tick_cnt, 1
   rjmp CHECK
PH3:
   cpi tick_cnt, 4

CHECK:
   brne MAIN          ; jeszcze nie koniec tej fazy

   ; koniec fazy: toggle LED i przejście do następnej
   clr tick_cnt
   sbi PINC, 0        ; toggle LED przez zapis do PINx

   inc phase
   cpi phase, 4
   brlo MAIN
   clr phase
   rjmp MAIN
```

ASM Example CTC Mode with Interrupt

```
.org TIMER1_COMPA_vect
   rjmp TIMER1_COMPA_ISR
.def temp = r16
.def flag = r17

.org 0x0000
   rjmp RESET

.org TIMER1_COMPA_vect
TIMER1_COMPA_ISR:
   ldi flag, 1
   reti

RESET:
   ; stack
   ldi temp, HIGH(RAMEND)
   out SPH, temp
   ldi temp, LOW(RAMEND)
   out SPL, temp

   ; LED output
   sbi DDRC, 0
   clr flag

   ; CTC mode, prescaler 256
   clr temp
   out TCCR1A, temp
   ldi temp, (1<<WGM12)|(1<<CS12)
   out TCCR1B, temp

   ; OCR1A = 6249
   ldi temp, HIGH(0x1869)
   out OCR1AH, temp
   ldi temp, LOW(0x1869)
   out OCR1AL, temp

   ; enable compare interrupt
   ldi temp, (1<<OCIE1A)
   out TIMSK1, temp

   sei

MAIN:
   tst flag
   breq MAIN

   clr flag
   sbi PINC, 0
   rjmp MAIN
```

When writing an Interrupt Service Routine (ISR):

Keep it *short*

- Don't use delay ()
- Don't do serial prints
- Make variables shared with the main code **volatile**
- Variables shared with the main code may need to be protected by "critical sections."
- Don't try to turn interrupts off or on

What are "volatile" variables?

Variables **shared** between ISR functions, and standard procedures should be declared "volatile". This tells the compiler that such variables might change at any time. Thus the compiler must reload the variable whenever you reference it, rather than relying upon a copy it might have in a processor register.

For example:

```
// global variable to count the number of overflows
volatile uint8_t tot_overflow;

// Interrupt Service Routine (ISR)
ISR(TIMER1_OVF_vect)
{
        // keep a track of number of overflows
        TCNT1L= 0x71;
        TCNT1H= 0x02;
        tot_overflow++;
}

int main ()
{
...
        while(1)
        {
                // check if no. of overflows = 12
                if (tot_overflow >= 1)  // NOTE: '>=' is used
                {
                        // check if the timer count reaches 53
                        PORTB ^= (1 << PB7);    // toggles the led
                        tot_overflow = 0;     // reset overflow counter
                }
        }
...
}  // end of loop
```

How long does it take to execute an ISR?

According to the datasheet, the minimal amount of time to service an interrupt is four clock cycles (to push the current program counter onto the stack) followed by the code now executing at the interrupt vector location. This usually contains a jump to the interrupt routine, which is another three cycles.

BIBLIOGRAPHY

1. Jarosław Doliński - "Mikrokontrolery AVR w praktyce"
2. Andrzej Pawluczuk - "Sztuka programowania mikrokontrolerów. AVR - podstawy"
3. Piotr Górecki - "Mikrokontrolery dla początkujących"
4. http://www.itee.uq.edu.au/~cse/_atmel/AVR_Studio_Tutorial/
5. http://winavr.scienceprog.com/avr-gcc-tutorial/
6. http://imakeprojects.com/Projects/avr-tutorial/
7. http://www.atmel.com/products/avr/
8. http://www.avrfreaks.net/
9. http://pl.wikipedia.org/wiki/Atmel_AVR
10. http://www.elportal.pl/ea/asm_avr.html
11. Jakub Jankowski, Marcin Kania, Mariusz Macheta, Łukasz Strzelecki – Opracowanie na temat mikrokontrolery AVR"
12. http://maxembedded.com/2011/06/avr-timers-timer1/

SCHEMATICS